

Анимация

Программная анимация

Основы анимаций

Наличие встроенных возможностей анимации представляют одну из ключевых особенностей платформы WPF. Анимации в WPF - это действительно мощное средство, которое при этом очень легко использовать. Но перед тем, как перейти к созданию анимаций, сразу надо сказать об ограничениях:

Одна анимация выполняется только для одного свойства зависимостей

Для анимации свойства необходим класс анимации, который бы поддерживал тип этого свойства. Например, для изменения таких свойств, как длина, ширина, которые представляют тип `double`, предназначен класс `DoubleAnimation`. Для изменения цвета фона или шрифта - `ColorAnimation`, для изменения свойства `Margin` - `ThicknessAnimation`.

За анимацию в WPF отвечает пространство имен `System.Windows.Media.Animation`. Оно содержит довольно большой набор классов, позволяющих анимировать различные свойства. Но в реальности все классы анимаций можно разделить условно на три группы:

TypeAnimation

Классы, которые производят линейную интерполяцию значений, благодаря чему при анимации свойство плавно изменяет свое значение. Как правило, такие классы называются по шаблону TypeAnimation, где Type - тип данных, которое представляет анимируемое свойства, например, DoubleAnimation:

- ByteAnimation
- ColorAnimation
- DecimalAnimation
- DoubleAnimation
- Int16Animation
- Int32Animation
- Int64Animation
- PointAnimation
- Point3DAnimation
- QuaternionAnimation
- RectAnimation
- Rotation3DAnimation
- SingleAnimation
- SizeAnimation
- ThicknessAnimation
- VectorAnimation
- Vector3DAnimation

PathGeometry

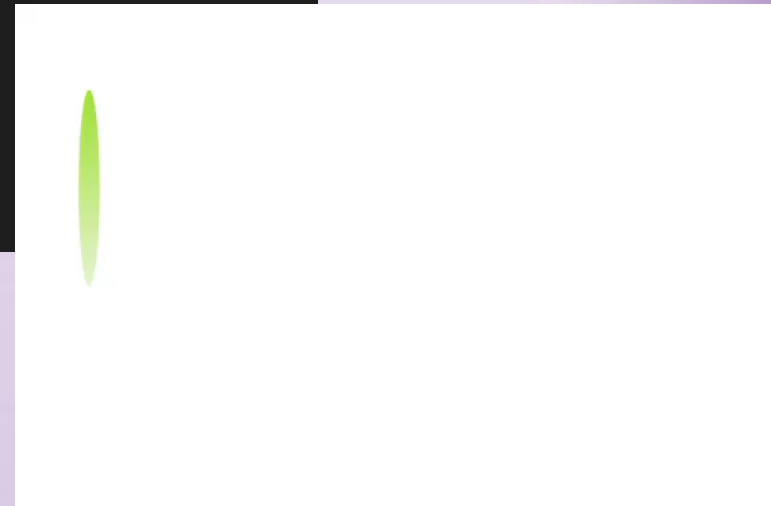
Классы, которые используют для анимации объект PathGeometry. Такие классы, как правило, называются по шаблону TypeAnimationUsingPath, например, DoubleAnimationUsingPath:

- DoubleAnimationUsingPath
- MatrixAnimationUsingPath
- PointAnimationUsingPath

```
Title="Анимация" Height="450" Width="800" Loaded="Window_Loaded">
<Grid>
    <Ellipse x:Name="el1" HorizontalAlignment="Left" Height="124" Margin="40,54,0,0"
        VerticalAlignment="Top" Width="143">
        <Ellipse.Fill>
            <LinearGradientBrush EndPoint="0.5,1" StartPoint="0.5,0"...>
        </Ellipse.Fill>
    </Ellipse>
</Grid>
```

```
private void Window_Loaded(object sender, RoutedEventArgs e)
{
    DoubleAnimation buttonAnimation = new DoubleAnimation();
    buttonAnimation.From = 10;
    buttonAnimation.To = 150;
    buttonAnimation.Duration = TimeSpan.FromSeconds(3);
    el1.BeginAnimation(Ellipse.WidthProperty, buttonAnimation);
}
```

buttonAnimation.RepeatBehavior =
RepeatBehavior.Forever; // добавить для
непрерывной анимации
buttonAnimation.AutoReverse = true; // для
плавного возвращения



Программная анимация

В данном случае мы сначала задаем тип анимации и создаем ее. Поскольку изменяется свойство Width, это будет DoubleAnimation.

У любого класса линейной анимации есть набор свойств, с помощью которых мы можем управлять анимацией:

From: начальное значение, с которого будет начинаться анимация

To: конечное значение

Duration: время анимации в виде объекта TimeSpan

By: значение, которое указывает, насколько должно увеличиться анимируемое свойство. Свойство By используется вместо свойства **To**

RepeatBehavior: позволяет установить, как анимация будет повторяться

AccelerationRatio: задает ускорение анимации

DecelerationRatio: устанавливает замедление анимации

SpeedRatio: устанавливает скорость анимации. По умолчанию значение 1.0

AutoReverse: при значении true анимация выполняется в противоположную сторону

FillBehavior: определяет поведение после окончания анимации. Если оно имеет значение Stop, то после окончания анимации объект возвращает прежние значения: `buttonAnimation.FillBehavior = FillBehavior.Stop`. Если же оно имеет значение HoldEnd, то анимация присваивает анимируемому свойству новое значение.

Чтобы запустить анимацию, у элемента вызывается метод `BeginAnimation()`. В этот метод передается свойство зависимости, которое надо анимировать, и сам объект анимации.

В данном случае произойдет изменение ширины кнопки от текущего значения до 150 пикселей. И данное изменение будет длиться 3 секунды.

Если мы хотим повторения анимации, можно использовать свойство `RepeatBehavior`:

```
1 DoubleAnimation buttonAnimation = new DoubleAnimation();
2 buttonAnimation.From = helloButton.ActualWidth;
3 buttonAnimation.To = 150;
4 buttonAnimation.Duration = TimeSpan.FromSeconds(3);
5 buttonAnimation.RepeatBehavior = new RepeatBehavior(2);
6 helloButton.BeginAnimation(Button.WidthProperty, buttonAnimation);
```

В данном случае анимация будет повторяться два раза. Также мы можем задать время повторения:

```
1 buttonAnimation.Duration = TimeSpan.FromSeconds(3);
2 buttonAnimation.RepeatBehavior = new RepeatBehavior(TimeSpan.FromSeconds(7));
```

Здесь время повторения - 7 секунд. Анимация длится 3 секунды, а это значит, что будет $7 / 3$ повторений: два полноценных повторения и в последнем случае ширина увеличится только до трети требуемой ширины.

Чтобы задать плавное изменение свойства в обратную сторону, применим свойство `AutoReverse`:

```
1 buttonAnimation.AutoReverse = true;
2 buttonAnimation.RepeatBehavior = new RepeatBehavior(5);
```


Событие Completed

При завершении анимации генерируется событие Completed, которое мы можем обработать:

```
1 public MainWindow()  
2 {  
3     InitializeComponent();  
4  
5     DoubleAnimation buttonAnimation = new DoubleAnimation();  
6     buttonAnimation.From = helloButton.ActualWidth;  
7     buttonAnimation.To = 150;  
8     buttonAnimation.Duration = TimeSpan.FromSeconds(5);  
9     buttonAnimation.Completed += ButtonAnimation_Completed;  
10    helloButton.BeginAnimation(Button.WidthProperty, buttonAnimation);  
11 }  
12  
13 private void ButtonAnimation_Completed(object sender, EventArgs e)  
14 {  
15     MessageBox.Show("Анимация завершена");  
16 }
```

Анимация нескольких свойств

```
ссылка:
private void Window_Loaded(object sender, RoutedEventArgs e)
{
    // анимация для ширины
    DoubleAnimation widthAnimation = new DoubleAnimation();
    widthAnimation.From = e1.ActualWidth;
    widthAnimation.To = 60;
    widthAnimation.Duration = TimeSpan.FromSeconds(5);

    // анимация для высоты
    DoubleAnimation heightAnimation = new DoubleAnimation();
    heightAnimation.From = e1.ActualHeight;
    heightAnimation.To = 60;
    heightAnimation.Duration = TimeSpan.FromSeconds(5);

    e1.BeginAnimation(Button.WidthProperty, widthAnimation);
    e1.BeginAnimation(Button.HeightProperty, heightAnimation);
}
```



Анимация других элементов и свойств

<Canvas>

<Button x:Name="btn" Width="200" Height="50" Content="Летающая кнопка">

<Button.Background>

<LinearGradientBrush EndPoint="0.5,1" StartPoint="0.5,0">

<GradientStop Color="■" "#FFFEFF95"/>

<GradientStop Color="■" "#FFC250BF" Offset="1"/>

</LinearGradientBrush>

</Button.Background>

</Button>

</Canvas>

```
DoubleAnimation widthAnimation = new DoubleAnimation();  
widthAnimation.From = 0;  
widthAnimation.To = 500;  
widthAnimation.Duration = TimeSpan.FromSeconds(5);  
widthAnimation.RepeatBehavior = RepeatBehavior.Forever;
```

```
DoubleAnimation heightAnimation = new DoubleAnimation();  
heightAnimation.From = 0;  
heightAnimation.To = 200;  
heightAnimation.Duration = TimeSpan.FromSeconds(5);  
heightAnimation.RepeatBehavior = RepeatBehavior.Forever;
```

```
btn.BeginAnimation(Canvas.LeftProperty, widthAnimation);  
btn.BeginAnimation(Canvas.TopProperty, heightAnimation);
```

Анимация других элементов и свойств

```
DoubleAnimation heightAnimation = new DoubleAnimation();  
heightAnimation.From = 25;  
heightAnimation.To = 200;  
heightAnimation.Duration = TimeSpan.FromSeconds(5);  
heightAnimation.RepeatBehavior = RepeatBehavior.Forever;  
  
rtBtn.BeginAnimation(RotateTransform.AngleProperty, heightAnimation);
```

```
<Button x:Name="btn" Width="200" Height="50" Content="Летающая кнопка"  
        Canvas.Left="59" Canvas.Top="96" RenderTransformOrigin="0.5,0.5">  
    <Button.RenderTransform>  
        <TransformGroup>  
            <ScaleTransform/>  
            <SkewTransform/>  
            <RotateTransform Angle="25.461" x:Name="rtBtn"/>  
            <TranslateTransform/>  
        </TransformGroup>  
    </Button.RenderTransform>
```

Анимация XAML

Анимация в XAML

Для определения анимации в XAML применяется объект **EventTrigger** или триггер событий. Этот объект имеет свойство **Actions**, которое определяет ряд действий, возникающих в результате генерации события. Само возникающее действие описывается через элемент **BeginStoryboard**, который и запускает анимацию.

Непосредственно для определения анимации используется объект **Storyboard**. Он объявляет объект анимации со всеми ее свойствами и параметрами. Например, проанимируем ширину кнопки:

```
1 <Window x:Class="AnimationApp.MainWindow"
2     xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
3     xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
4     xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
5     xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
6     xmlns:local="clr-namespace:AnimationApp"
7     mc:Ignorable="d"
8     Title="MainWindow" Height="250" Width="300">
9     <Window.Triggers>
10         <EventTrigger RoutedEvent="Loaded">
11             <EventTrigger.Actions>
12                 <BeginStoryboard>
13                     <Storyboard TargetProperty="Width" TargetName="helloButton">
14                         <DoubleAnimation From="70" To="150"
15                             AutoReverse="True"
16                             RepeatBehavior="0:0:10"
17                             Duration="0:0:3"
18                             Completed="ButtonAnimation_Completed" />
19                     </Storyboard>
20                 </BeginStoryboard>
21             </EventTrigger.Actions>
22         </EventTrigger>
23     </Window.Triggers>
24     <Grid>
25         <Button x:Name="helloButton" Width="70" Height="30" Content="Hello" />
26     </Grid>
27 </Window>
```


У объекта **EventTrigger** с помощью атрибута **RoutedEvent** определяется событие, которое будет запускать анимацию. В данном случае это событие **Loaded** - загрузка окна.

Ряд настроек анимации устанавливает элемент **Storyboard: TargetName** задает анимируемый элемент, а **TargetProperty** определяет свойство элемента, которое будет анимироваться.

В принципе эти настройки также можно было бы вынести в объект анимации в виде прикрепляемых свойств:

```
<Storyboard>
```

```
  <DoubleAnimation Storyboard.TargetProperty="Width" Storyboard.TargetName="helloButton"
```

```
    From="70" To="150" ...>
```

И далее в самом объекте **Storyboard** определяется объект анимации **DoubleAnimation** с рядом настроек. Все настройки в принципе тут те же, что использовались для анимации в прошлой теме. Правда, при определении значений свойств здесь есть некоторые отличия.

Так, свойство **RepeatBehavior** инициализируется временем - "0:0:10", которое будет повторяться анимация. Чтобы указать число повторов, структуру **RepeatBehavior** надо инициализировать так: **RepeatBehavior="2x"**, где 2 - количество повторов, а x - просто префикс, указывающий, что речь идет о количестве итераций, без него бы число интерпретировалось как количество дней. Третий способ задания этого свойства - **RepeatBehavior="Forever"** - в этом случае анимация будет продолжаться все время работы приложения.

Например, ниже установлено **RepeatBehavior="Forever"**, а свойство **DecelerationRatio** замедляет анимацию, что создает эффект подскока шара в верх, где он, достигая максимальной точки, теряет скорость, а при падении вновь ее увеличивает:

```
8      Title="MainWindow" Height="250" Width="300">
9      <Window.Triggers>
10         <EventTrigger RoutedEvent="Button.Click">
11             <EventTrigger.Actions>
12                 <BeginStoryboard>
13                     <Storyboard Timeline.DesiredFrameRate="60">
14                         <DoubleAnimation Storyboard.TargetName="ball" Storyboard.TargetProperty="(Canvas.Bottom)"
15                             From="0" To="160" AutoReverse="True" Duration="0:0:2.5" RepeatBehavior="Forever"
16                             DecelerationRatio="1"/>
17                     </Storyboard>
18                 </BeginStoryboard>
19             </EventTrigger.Actions>
20         </EventTrigger>
21     </Window.Triggers>
22     <Grid>
23         <Grid.RowDefinitions>
24             <RowDefinition />
25             <RowDefinition Height="Auto" />
26         </Grid.RowDefinitions>
27         <Canvas Background="LightPink">
28             <Ellipse Name="ball" Fill="Red" Stroke="Black" Width="15" Height="15"
29                 Canvas.Left="130" Canvas.Bottom="0" />
30         </Canvas>
31         <Button Width="70" Height="25" Content="Кнопка" Grid.Row="1" Margin="10" />
32     </Grid>
33 </Window>
```

В данном случае так как в EventTrigger в качестве события определено Button.Click, то анимация будет запускаться по нажатию на кнопку.

По умолчанию анимация вызывается 60 раз в секунду, однако с помощью прикрепленного свойства Timeline.DesiredFrameRate можно задать частоту кадров в секунду, как в предыдущем примере.

Анимация свойств вложенных объектов

Анимация может применяться и к свойствам вложенных объектов, которые являются свойствами, например:

```
1 <Ellipse Name="ball" Stroke="Black" Width="20" Height="20" Canvas.Left="130" Canvas.Bottom="0">
2   <Ellipse.Fill>
3     <RadialGradientBrush RadiusX="1" RadiusY="1" GradientOrigin="0.3, 0.3">
4       <GradientStop Color="White" Offset="0" />
5       <GradientStop Color="Blue" Offset="1" />
6     </RadialGradientBrush>
7   </Ellipse.Fill>
8   <Ellipse.Triggers>
9     <EventTrigger RoutedEvent="Window.Loaded">
10      <BeginStoryboard>
11        <Storyboard>
12          <ColorAnimation Storyboard.TargetProperty="Fill.GradientStops[1].Color"
13            To="Yellow" Duration="0:0:8" AutoReverse="True"
14            RepeatBehavior="Forever" />
15        </Storyboard>
16      </BeginStoryboard>
17    </EventTrigger>
18  </Ellipse.Triggers>
19 </Ellipse>
```

Здесь свойство **Fill** элемента **Ellipse** инициализируется кистью **RadialGradientBrush**, которая имеет коллекцию **GradientStops**. Анимация же применяется ко второму объекту коллекции и его свойству **Color**.

Комплексные анимации

С помощью объекта Storyboard можно создавать и более комплексные анимации. Например, сделаем кнопку, которая одновременно меняет ширину, длину и цвет:

```
1 <Button x:Name="helloButton" Foreground="White" Width="70" Height="25" Content="Кнопка">
2     <Button.Background>
3         <SolidColorBrush x:Name="buttonColor" Color="Black" />
4     </Button.Background>
5     <Button.Triggers>
6         <EventTrigger RoutedEvent="Loaded">
7             <EventTrigger.Actions>
8                 <BeginStoryboard>
9                     <Storyboard>
10                        <DoubleAnimation Storyboard.TargetProperty="Width" Storyboard.TargetName="helloButton"
11                            From="80" To="150" AutoReverse="True" RepeatBehavior="0:0:10" Duration="0:0:2" />
12                        <DoubleAnimation Storyboard.TargetProperty="Height" Storyboard.TargetName="helloButton"
13                            From="30" To="100" AutoReverse="True" RepeatBehavior="0:0:10" Duration="0:0:2" />
14                        <ColorAnimation Storyboard.TargetName="buttonColor" Storyboard.TargetProperty="Color"
15                            From="{Binding ElementName=buttonColor, Path=Color}" To="Red"
16                            AutoReverse="True" RepeatBehavior="0:0:10" Duration="0:0:2" />
17                    </Storyboard>
18                </BeginStoryboard>
19            </EventTrigger.Actions>
20        </EventTrigger>
21    </Button.Triggers>
22 </Button>
```

Доп. Информация про анимацию

- <https://metanit.com/sharp/wpf/16.3.php>
- <https://metanit.com/sharp/wpf/16.4.php>
- <https://metanit.com/sharp/wpf/16.5.php>
- <https://learn.microsoft.com/ru-ru/dotnet/desktop/wpf/graphics-multimedia/animation-overview>

Задание

Анимировать двух животных:

1. Одно животное анимировать в xaml, второе программно
2. Анимировать передвижение использовать `Canvas.Left/Bottom/Right/Top`
3. Анимировать две части тела: например ноги, голова или хвост

Анимация частей тела должна изменять широту/высоту и `TransformGroup` (`ScaleTransform/ SkewTransform/ RotateTransform / TranslateTransform`).

Так же добавить `ColorAnimation` для какой-либо части тела

Сделать анимации непрерывной

